

INTRODUCTION TO DATA STRUCTURES and ALGORITHMS

Rutgers University

UNIT TESTING WITH JUNIT

- What is Unit Testing?
- Getting Started with JUnit
- Writing Tests in JUnit
- Assertions
- FreeShipping Example
- CatTest Example



In software development, **Unit Tests** are used to verify code works, piece by piece.

Unit tests verify that each part of code works as intended using specific test cases or conditions. They allow you to:

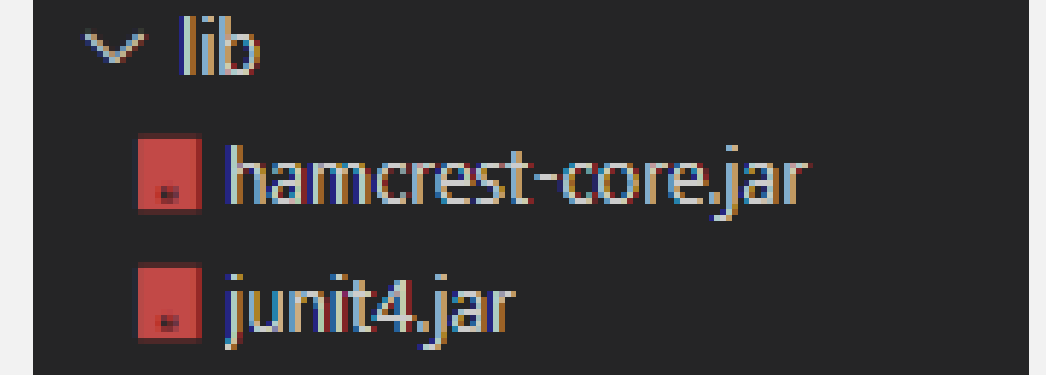
- Better understand how your code works
- Identify corner cases + boundaries in which your code can fail
- Spot bugs in smaller parts of code before they become larger issues
- See if changes in one part of the code affects other parts

Note: Unit tests are only as good as the conditions you test for.

To write Unit Tests in Java, we use a java testing framework called **JUnit**.

Getting Started with JUnit

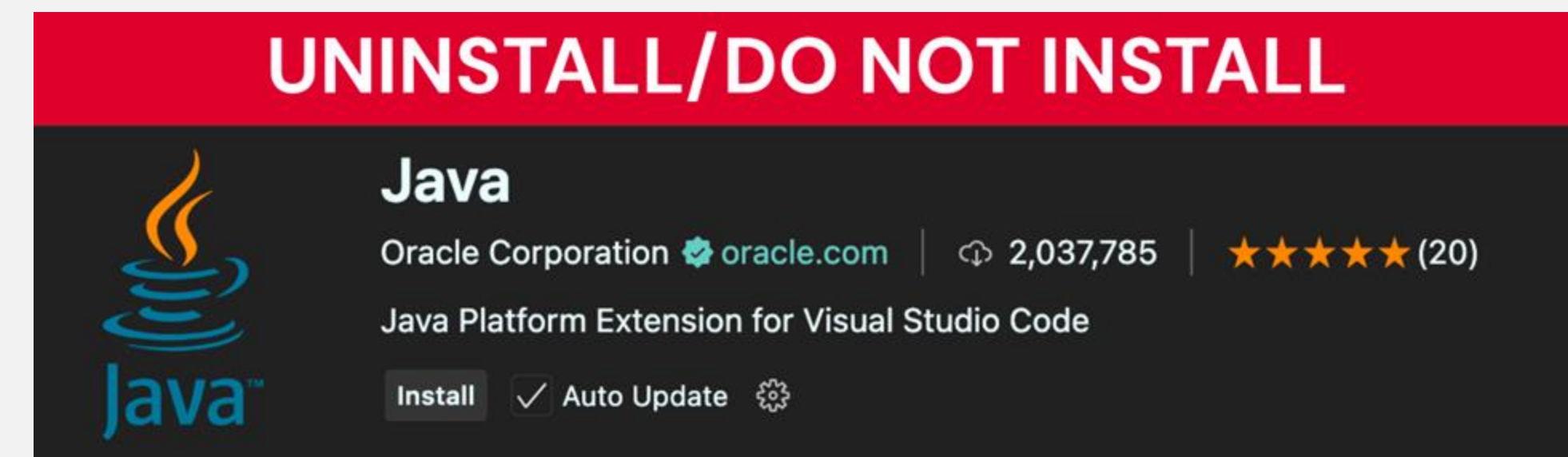
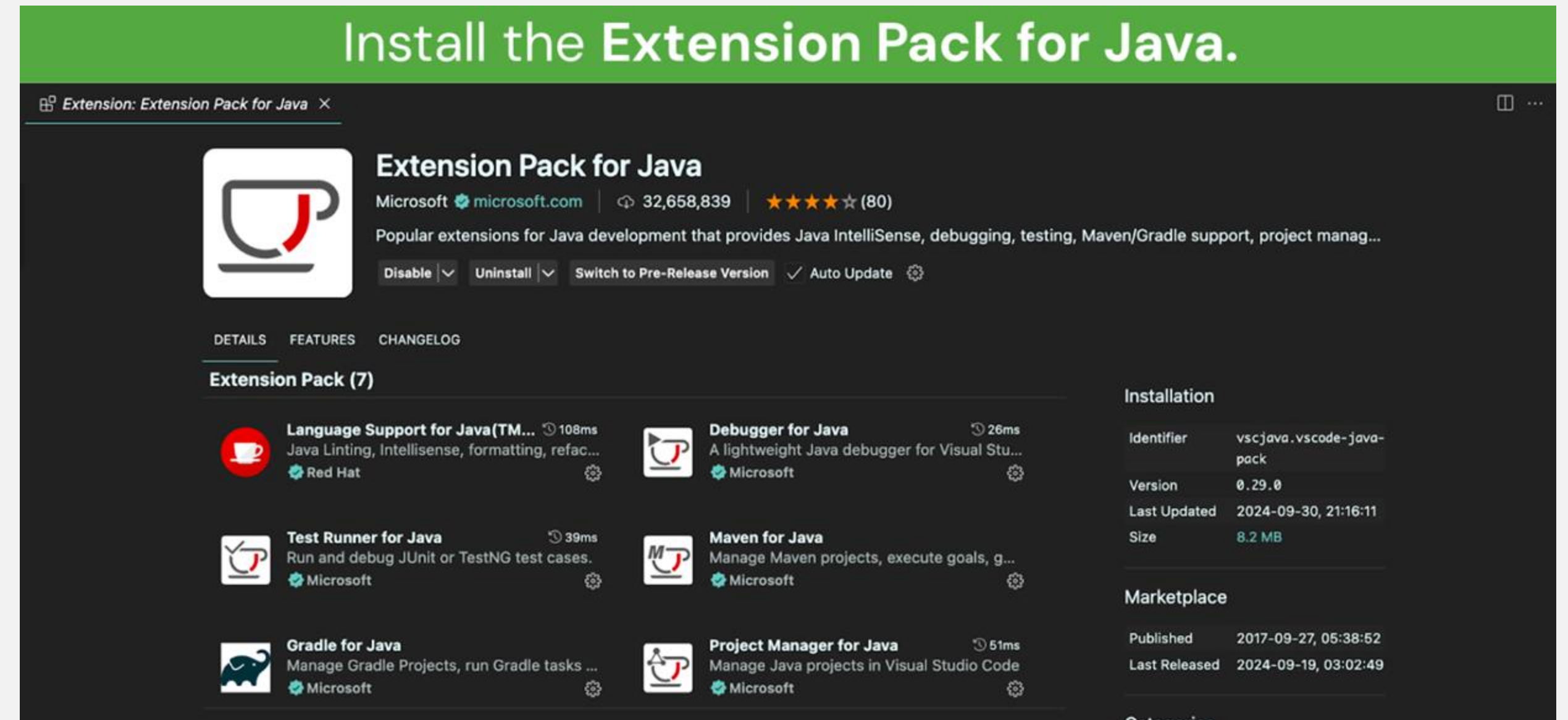
We use **JUnit 4** - this is included in the “lib” folder in all assignments/labs.



- Hit the Extensions menu on VS Code and search for “Extension Pack for Java”, then install.

- This should install “Test Runner For Java”, along with other extensions needed for course assignments.

- Note: If you’ve the Java extension by “Oracle Corporation” installed, **uninstall it.**



Writing Tests in JUnit

- To use **JUnit**, we write Tests as methods inside of a Test Class
 - Each test class tests a single Java class, and each test method tests a single thing in that class.
 - Test classes and their methods are named around what they test. Such as “*ClassTest.java*” and “*testMethod()*”.
 - Follow the same standards for all tests and be clear about what each test actually does.

- Test methods are *annotated* by the **@Test** tag, as shown in the example to the right.
 - This tells JUnit it is a test method.
 - Allows it to be run with the Test Runner extension.

```
import static org.junit.Assert.*;
```

```
import org.junit.*;
```

```
public class ExampleTest {
```

```
    @Test
```

```
    public void testExample() {
```

```
        // Empty test method
```

```
    }
```

```
}
```

We import the JUnit framework into our Test class with these statements

Assertions – enable you to test your assumptions

- Each test method is an independent block of code, where you can check if certain conditions are true. This allows you to test that your code is giving you the correct output, for any input you choose.
- We use *assertion* statements to do this. These are included in the **JUnit** framework.
- When the system runs an assertion, it evaluates the expression, and if it is false, it fails the test.

- Some assertion methods you can use are:
 - `assertTrue( boolean condition )` - checks to see if the given `condition == true`
 - `assertFalse( boolean condition )` - checks to see if the given `condition == false`
 - `assertEquals(expected, actual)` - checks to see if `expected.equals(actual)`

```
@Test
public void testMethodExample() {
    assertTrue(inputNumber >= 1);
    assertEquals("Cat", inputString);
}
```

“inputNumber” and “inputString” would get replaced by data from whatever class you are testing

- `testMethodExample()` will **FAIL** if `inputNumber` is `< 1` **OR** if `inputString` is NOT “Cat”

Example

- We can create a basic example to test a method which returns true if the age is eligible to vote.

```
package elections;

public class CanYouVote {
    public static boolean isEligible(int total) {
        if (total < 18 ) return false;
        return true;
    }
}
```

- Use the method *assertFalse* to test the cases where the age isn't allowed to vote, and *assertTrue* when they are.
- Always write tests for any boundaries, such as at any threshold numbers (18 for the voting age). As well as checking invalid inputs, like negative ages.

```
import static org.junit.Assert.*;
import org.junit.*;
import elections.*;

public class TestCanYouVote {
    @Test
    public void testIsEligible() {
        // Negative years can't vote
        assertFalse(CanYouVote.isEligible(-1));
        // Someone between 0-17 can't vote
        assertFalse(CanYouVote.isEligible(5));
        assertFalse(CanYouVote.isEligible(17));
        // Exactly 18? You can vote
        assertTrue(CanYouVote.isEligible(18));
        // Greater than 18? You can vote
        assertTrue(CanYouVote.isEligible(60));
    }
}
```

CatTest Example

- We can create tests for objects as well. We will test the Cat class from Lab 1, shown below.

```
package island;

public class Cat {
    private String name;
    private Island homeIsland;
    private Coordinate coords;
    private Color color;
    public static String species = "felis catus";

    public Cat(String name, Island home, int row, int col, Color color) {
        this.name = name;
        this.homeIsland = home;
        this.coords = new Coordinate(row, col);
        this.color = color;
        home.getTiles()[row][col].cat = this; // Place itself on island
    }
}
```

- Cat also has the following getter methods we will test:

```
public String getName()
public Color getColor()
public Island getIsland()
public int getRow()
public Coordinate getCol()
```

CatTest Example

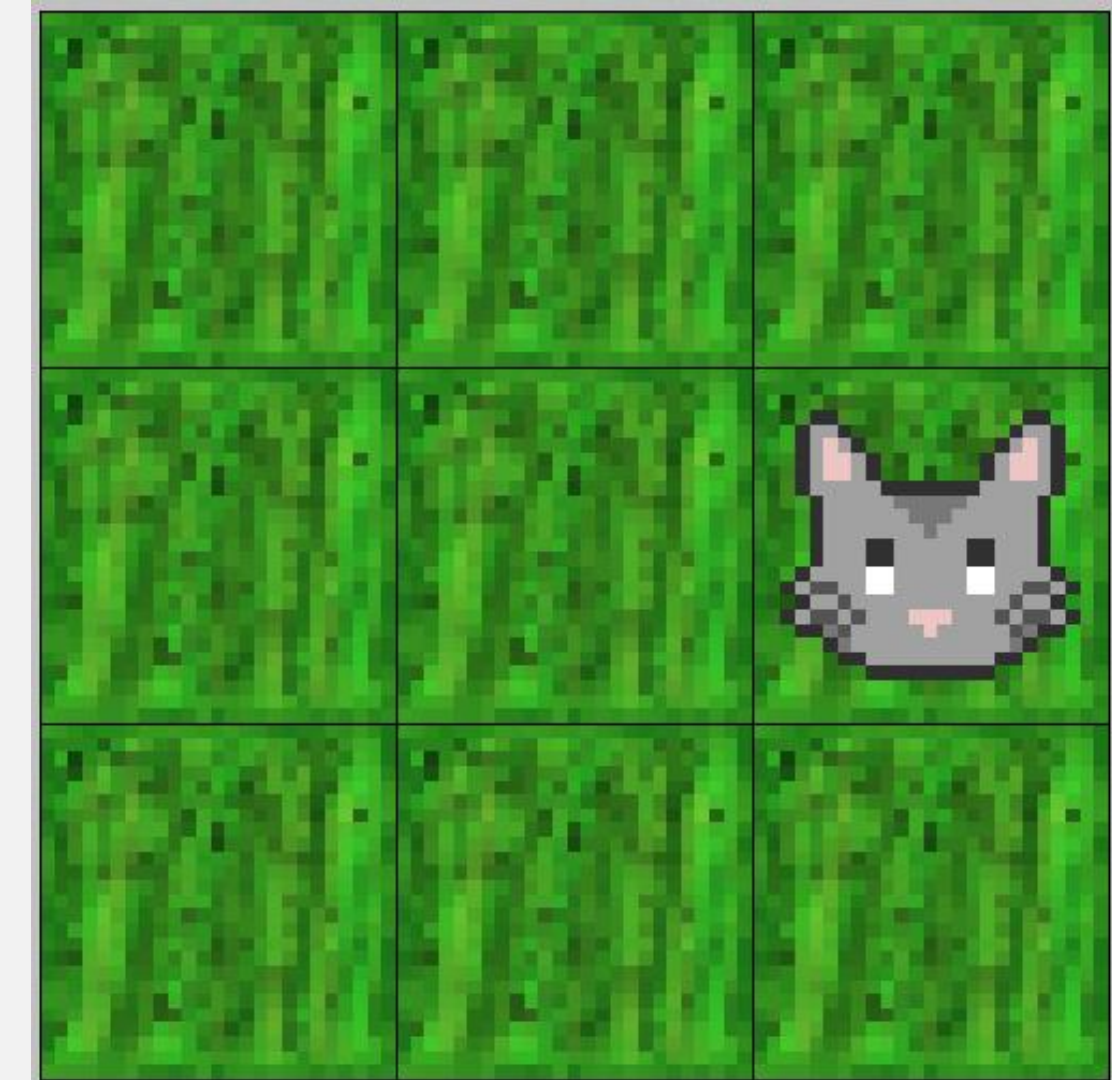
- This test will first create the Island and Cat objects, which initializes the cat's attributes.

```
import island.*;
import island.constants.*;
public class CatTest {
    @Test
    public void testCatGetters() {
        // 3x3 island of land tiles
        Island home = Island.createIsland(new String[][]{
            {"L", "L", "L"},
            {"L", "L", "L"},
            {"L", "L", "L"}
        });
        Cat meow = new Cat("Flo", home, 1, 2, Color.GREY);
        assertEquals(meow.getName(), "Flo"); // Test Name
        assertEquals(meow.getRow(), 1); // Test Row
        assertEquals(meow.getCol(), 2); // Test Col
        assertTrue(meow.getIsland() == home); // Test Island
        assertEquals(meow.getColor(), Color.GREY); // Test Color
    }
}
```

← package and JUnit imports omitted for space

← This creates a 3x3 island of land

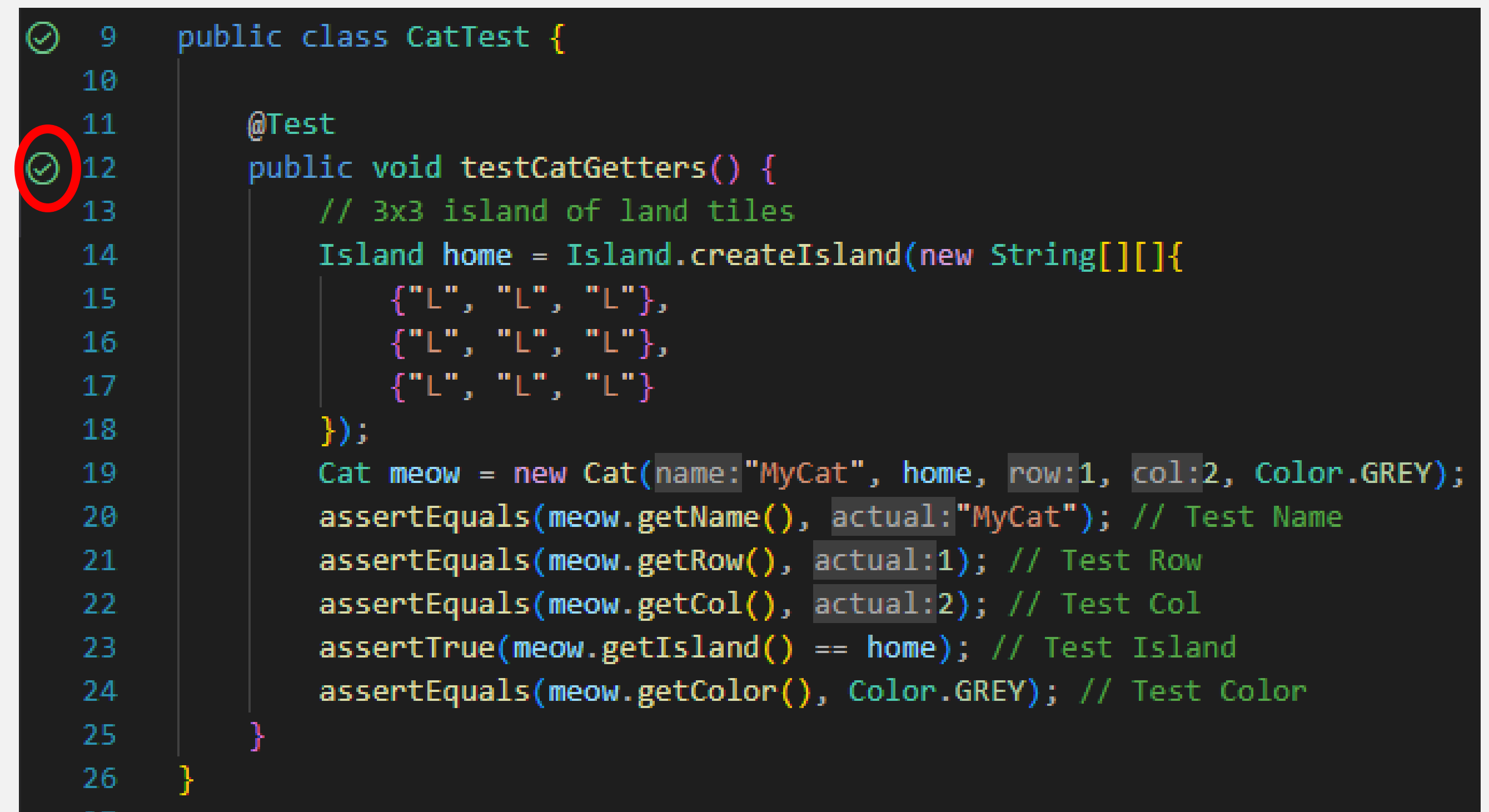
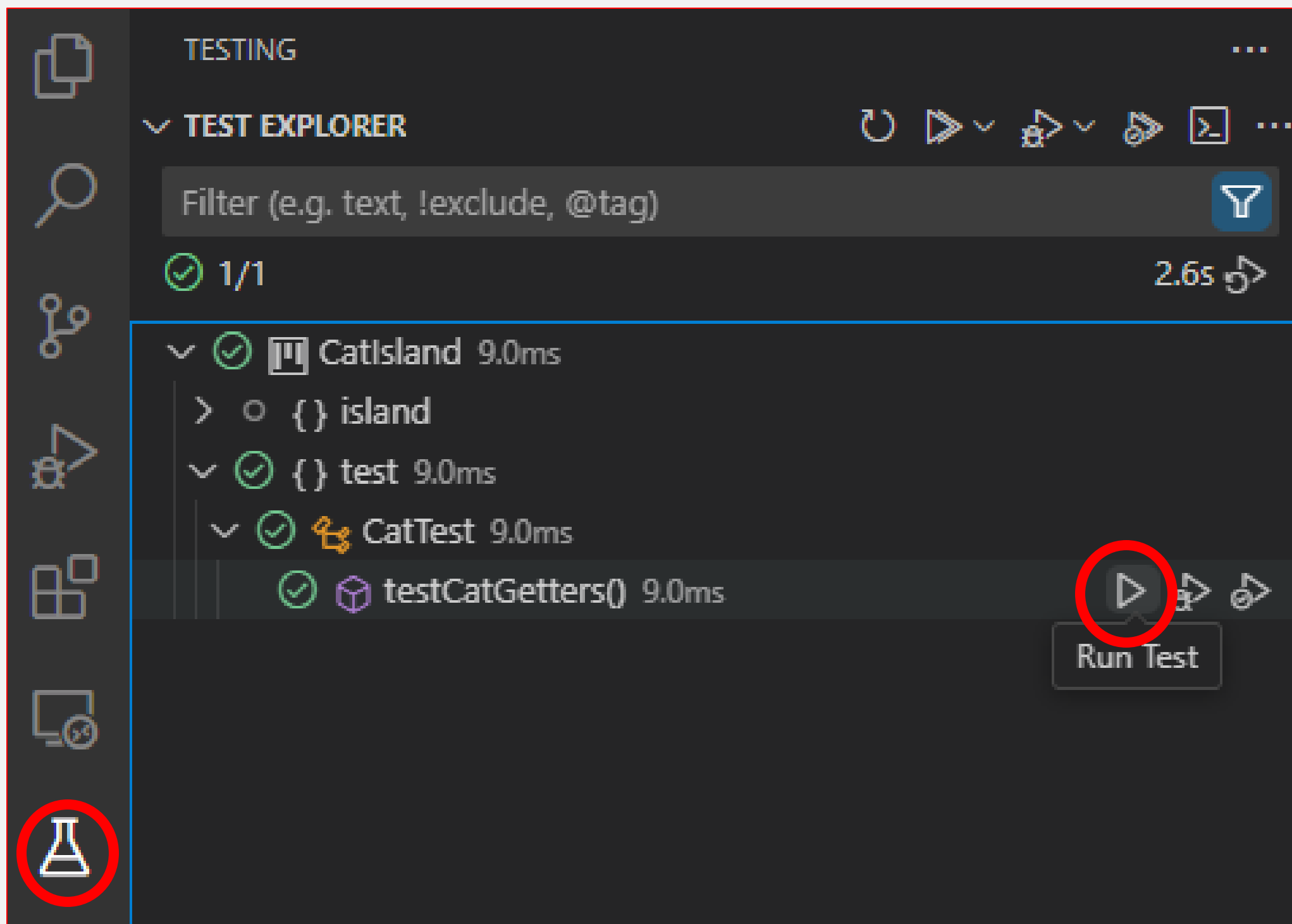
← This places a cat on that island



- We check the Cats name, row, column, island, and color, according to the values we expect based on the constructor.

Running CatTest

- We run tests through the Test Runner, found in the VSCode menu bar
 - You can also run in the code window itself.
- A green checkmark means passed. You can either run individual tests, or the whole class and check all tests at once.



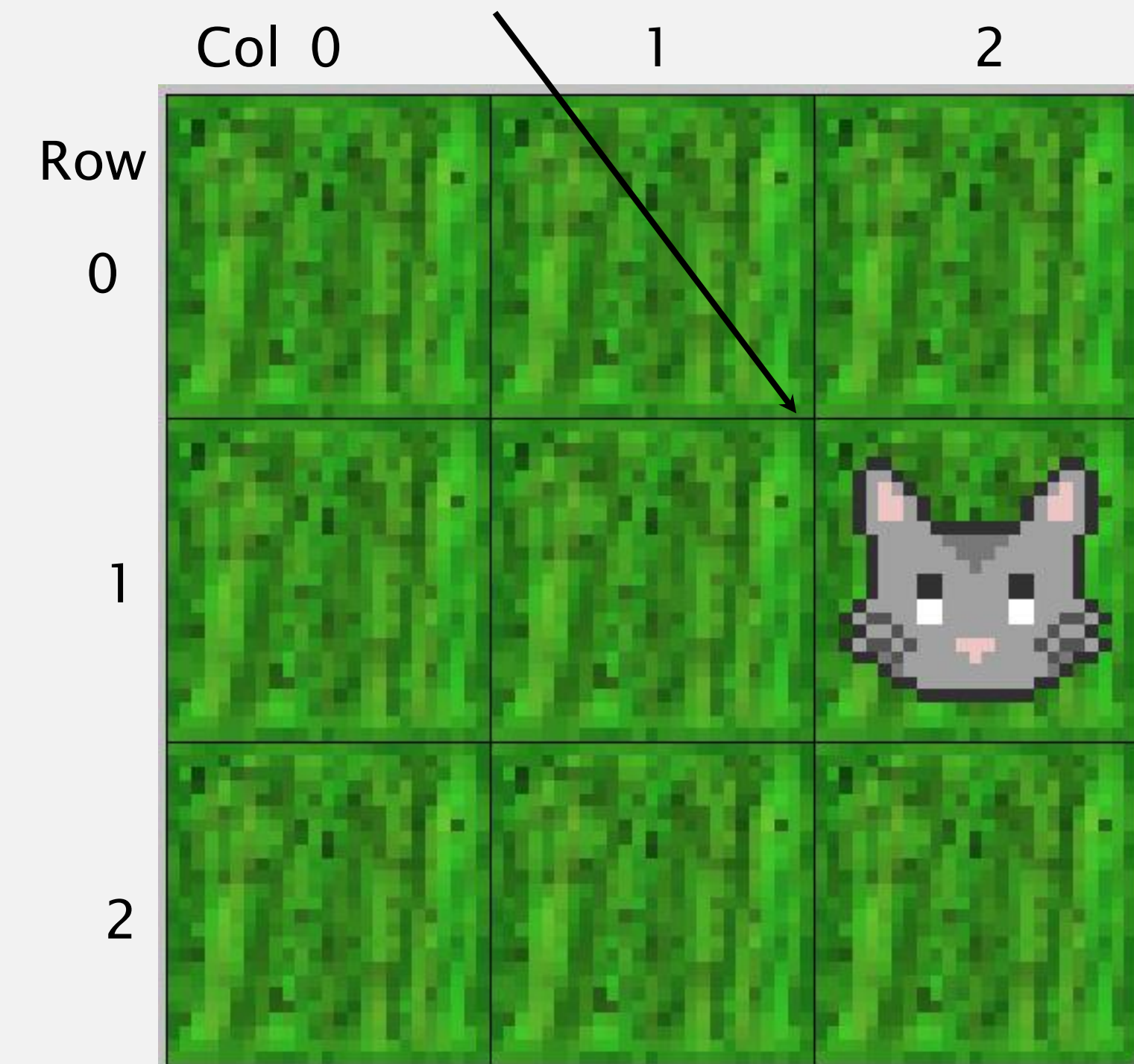
Incorrect CatTest Example

- Be careful writing test cases, as they can make you think a bug exists when it doesn't.

```
import island.*;
import island.constants.*;
public class CatTest {
    @Test
    public void testCatGetters() {
        // 3x3 island of land tiles
        Island home = Island.createIsland(new String[][]{
            {"L", "L", "L"},
            {"L", "L", "L"},
            {"L", "L", "L"}
        });
        Cat meow = new Cat("MyCat", home, 1, 2, Color.GREY);
        assertEquals(meow.getRow(), 2);
        assertEquals(meow.getCol(), 1);
    }
}
```

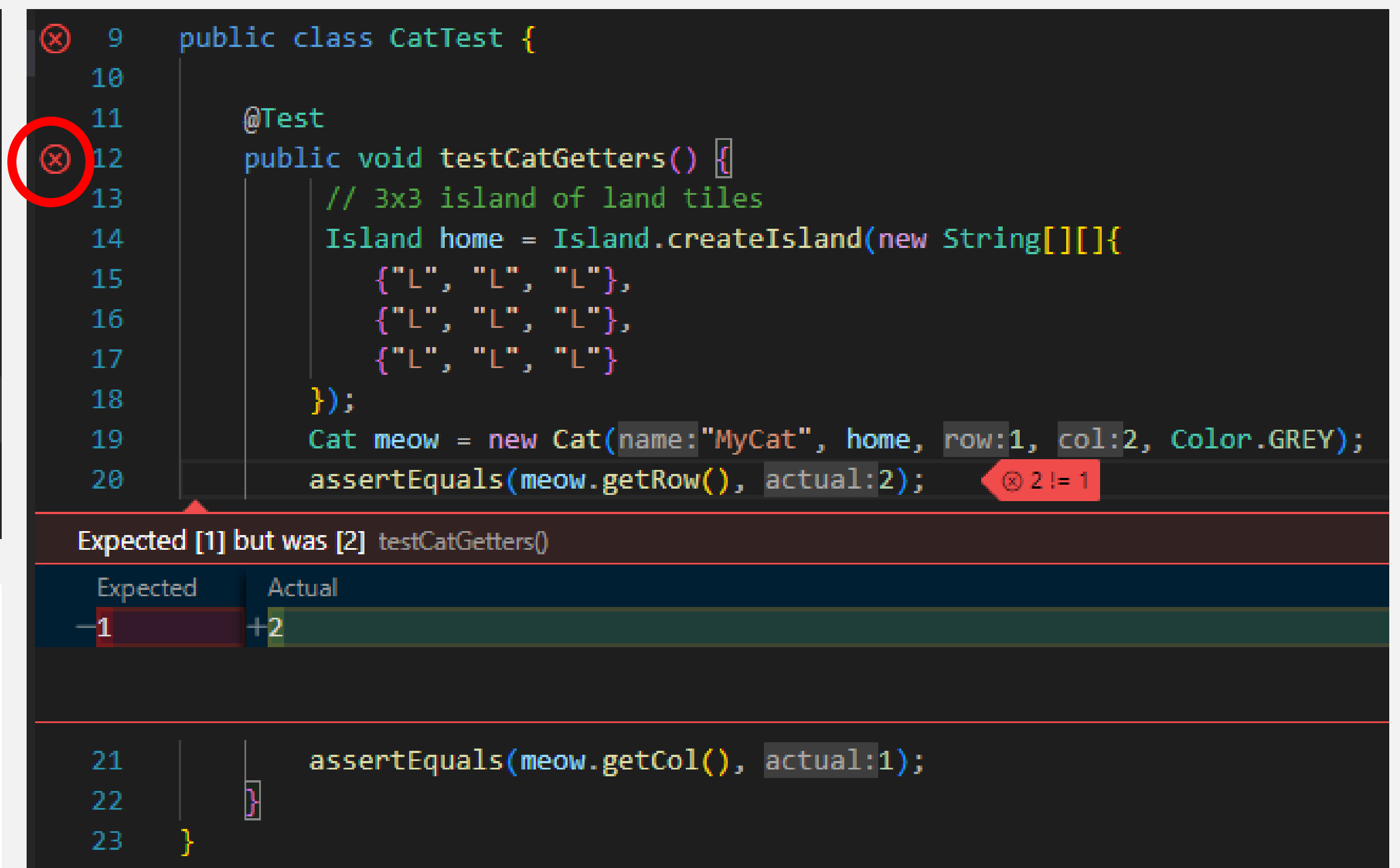
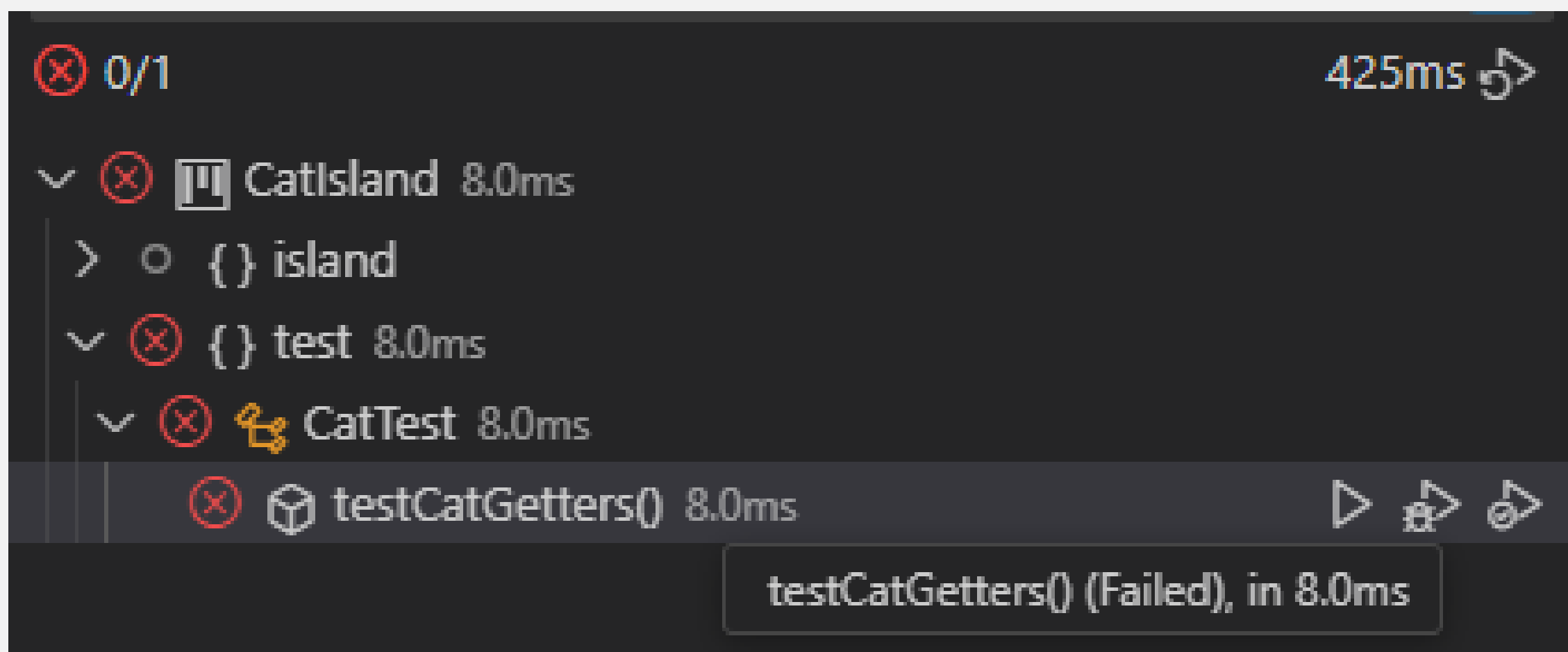
If we (incorrectly) swap the test cases for the attributes, the test will fail when it shouldn't

In reality, the cat is still at row 1 and column 2 (zero-indexed from top left corner)



Failing CatTest

- A red checkmark indicates that the test failed.
- A failed test will throw an AssertionError / ComparisonError, at the failed assert method.
 - A single failed assert method will fail the whole test method. It will not execute any further lines of that test.



- We are incorrectly testing that the row is 2 and column is 1, instead of the correct test where the row is 1 and column is 2.

Summary

Unit Tests are used to verify code works, piece by piece.

Every class is expected have its own dedicated Test class, each feature having its own test method.

JUnit tests are methods annotated with the **@Test** tag, which use ***assert...()*** methods
-assertEquals(), assertEquals(), assertTrue(), assertFalse(), assertNull(), assertNotNull(),
...

Test cases are only as good as you write them and can be subject to bugs just like other code.

For assignments, you'll often print out values contained in your complex structures. In your tests, you can then compare these printed values against the ***expected*** output.